

# Reflection – Homework Week 14: Date Display GUI 2

## 1. Main Functionalities and Implementation

In this homework, I redesigned the Date Display GUI application using JavaFX. The application allows users to select a base date, enter a number, choose a direction (Before or After), and select a time unit (Days, Months, or Years). After clicking the Apply button, the application calculates and displays the resulting date.

Another important feature is dynamic date formatting. Users can choose different date formats such as Vietnamese format (dd/MM/yyyy), US format (MM/dd/yyyy), European format (dd.MM/yyyy), and ISO format (yyyy-MM-dd). The displayed date updates automatically whenever the selected date or format changes.

The application uses Java's LocalDate class to perform date calculations accurately and handle date-related edge cases.

## 2. Design Decisions and MVC Architecture

The application follows the MVC (Model–View–Controller) architectural pattern.

- **Model:** The DateModel class stores the selected date, selected date format, and result date using JavaFX properties.
- **View:** The user interface is implemented directly using JavaFX components in the DateView class. It contains all controls such as DatePicker, TextField, ComboBox, RadioButton, Button, and Label.
- **Controller:** The DateController class handles user interactions, validates input, communicates with the model, and updates the view.

Using MVC separates the business logic from the graphical interface, making the program easier to understand, maintain, and extend.

## 3. Improvements Compared with the Previous Version

Compared with the previous AWT/Swing version, this JavaFX version is more modular and reactive.

The previous version relied more on manual updates of GUI components. In this version, property binding automatically updates displayed information when related data changes. The

code structure is also cleaner because the responsibilities are separated into different classes according to MVC principles.

Additionally, input validation has been improved. Invalid inputs such as empty values, non-numeric values, or negative numbers are detected immediately, and the Apply button is automatically disabled to prevent incorrect calculations.

#### **4. Use of JavaFX Properties, Binding, and Observable Collections**

JavaFX properties were used to store and observe changes in the selected date and result date.

Property binding was applied to the displayed date label so that it automatically updates whenever the selected date or date format changes. This improves user experience because the interface always stays synchronized with the underlying data.

Observable collections were used to manage the available date formats and time units displayed in the ComboBox controls. This makes the interface flexible and allows the available options to be managed efficiently.

#### **5. Conclusion**

This homework helped me gain a better understanding of JavaFX development, MVC architecture, property binding, and observable collections. Through this assignment, I learned how to build a more maintainable and responsive GUI application while following good object-oriented design principles.